



Chapter2: Memory, I/O addressing

Asst.Prof.Dr.Supakit Nootyaskool

Objective

- Tell the relationship between binary and hexadecimal.
- Understand parity and error correction.
- Know what is inside the text file.
- Understand address bus, data bus, and control signal
- Tell devices selecting on the decoder circuit.

Topics

- Relationship between binary and hexadecimal
- Parity and error correction
- Text data
- Address bus, Data bus, and Control signal
- Decoder

RELATIONSHIP BETWEEN BINARY AND HEXADECIMAL

Able to convert binary to hexadecimal and convert back.

When you read a computer code

```
#include <dos.h>
#include <stdio.h>
#include <conio.h>
#define PORT1 0x3F8 /* Defines Serial Port Base Address (COM1 */
void main(void){
    unsigned char c = 0;
    unsigned char chrctr = 0;
    /*int exit = 1; */
    outportb(PORT1 + 1, 0); /* Turn off interrupts */
    /* PORT1 Communication Settings */
    outportb(PORT1 + 3, 0x80); /* Set DLAB ON */
    outportb(PORT1 + 0, 0x0C); /* Set the baud rate to 9600 */
    outportb(PORT1 + 1, 0x00); /* Set Baud - Divisor latch HIGH */
    outportb(PORT1 + 3, 0x03); /* 8 bits, no parity, 1 stop */
    outportb(PORT1 + 2, 0xC7); /* FIFO Control Register */
    outportb(PORT1 + 4, 0x0B); /* Turn on DTR, RTS, and OUT2) */
    printf("Waiting on transmission from source.\nPress ESC to quit.\n");
    while(chrctr != 27){ /* Execute the loop if ESC has been hit */
        c = inportb(PORT1 + 5);
        if (c & 0x01){
            chrctr = inportb(PORT1);
            printf("%d",chrctr);
        }
        if (kbhit()){
            chrctr = getch();
            outportb(PORT1, chrctr);
        }
    }
}
```

What is 0x3F8?



Binary <-> Hexadecimal

0000

0

1000

8

0001

1

1001

9

0010

2

1010

A

0011

3

1011

B

0100

4

1100

C

0101

5

1101

D

0110

6

1110

E

0111

7

1111

F

Activity 2.1 Convert Binary and Hex

- 3F8H

0011-1111 - 1000

0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

- 01101100111001111100111101
D 9 C F 3 D

Format of the number system

- We can conclude that

$$\text{Value} \times \text{Base}^{\text{position}-1}$$

- $101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5 = ?_{10}$
- $453_8 = 256 + 40 + 3 = 299 = ?_{10}$
- $453_{16} = 1024 + 80 + 3 = 1107 = ?_{10}$

Relationship between binary, octal, and hexadecimal

- $4310_{10} = 1000011100001$
divided in 4 = 0001 0000 1110 0001
divided in 3 = 001 000 011 100 001

0001 0000 1110 0001

Hex = 1 0 E 1 = 10E1₁₆

001 000 011 100 001

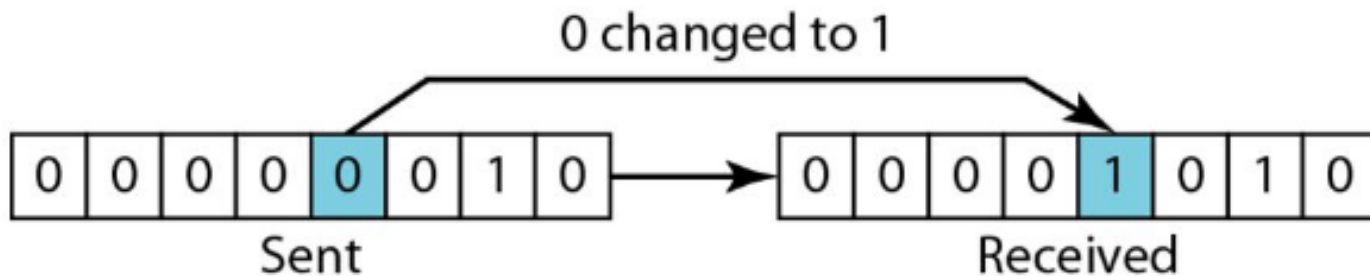
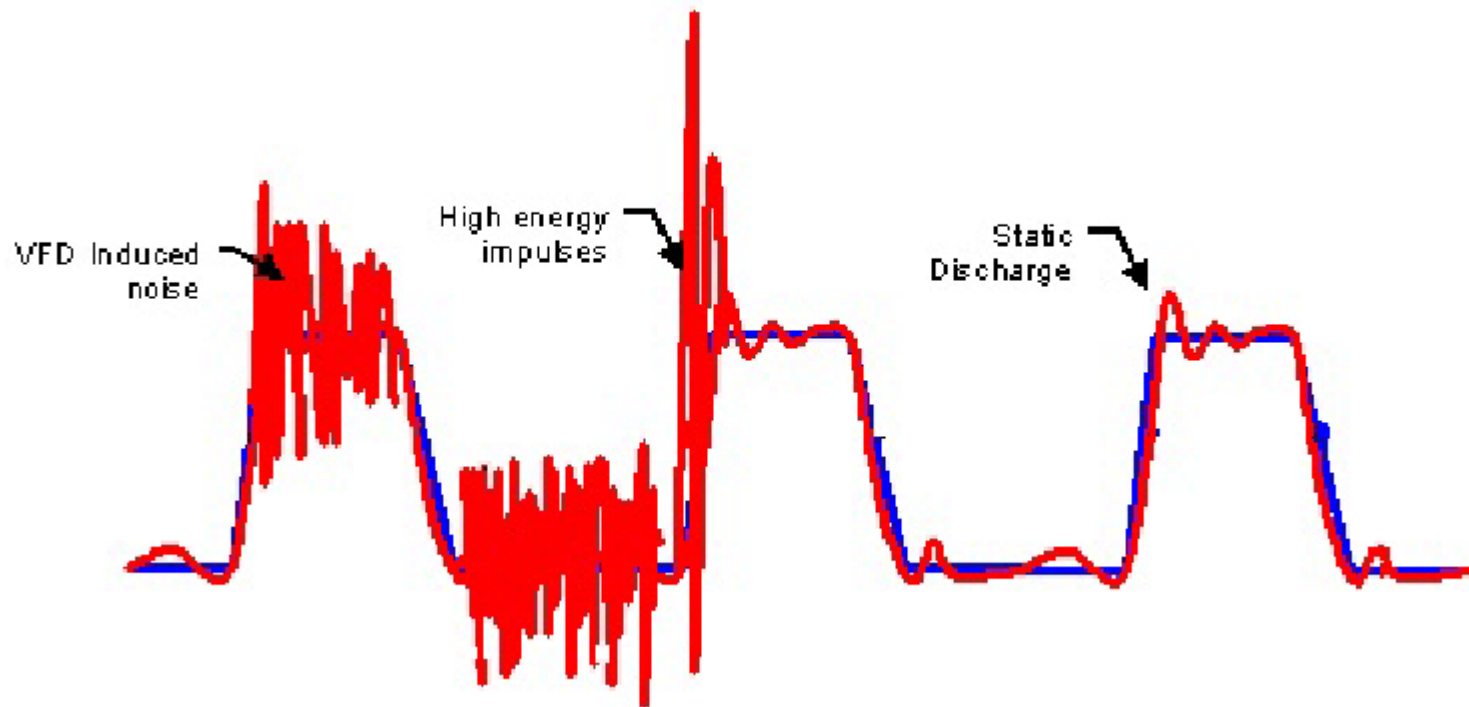
Oct = 1 0 3 4 1 = 10341₈

PARITY AND ERROR CORRECTION

Write odd and even parity

Tell error correction checking

Error in signal communication



พาริตีบิต ใช้สำหรับตรวจสอบข้อผิดพลาด

Parity bit and error correction

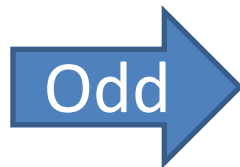
- A parity bit is a bit appended to a binary sequence in order to sum of the number in even or odd.

- Even = 0, Odd = 1

0100



0100***1***



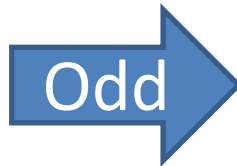
0100***0***

Parity bit and error correction

0110



0110*0*



0110*1*

001110

Handwritten annotations:
A blue line connects the first '1' to the second '1'.
A blue line connects the second '1' to the third '1'.
A blue line connects the third '1' to the '0'.
A pink dot is placed above the final '0'.
Below the first '1' is a blue scribble.
Below the second '1' is a blue '4'.
Below the third '1' is a blue '2'.



001110*1*



001110*0*

Activity 2.2 write parity bit

101101

Even → 1011010

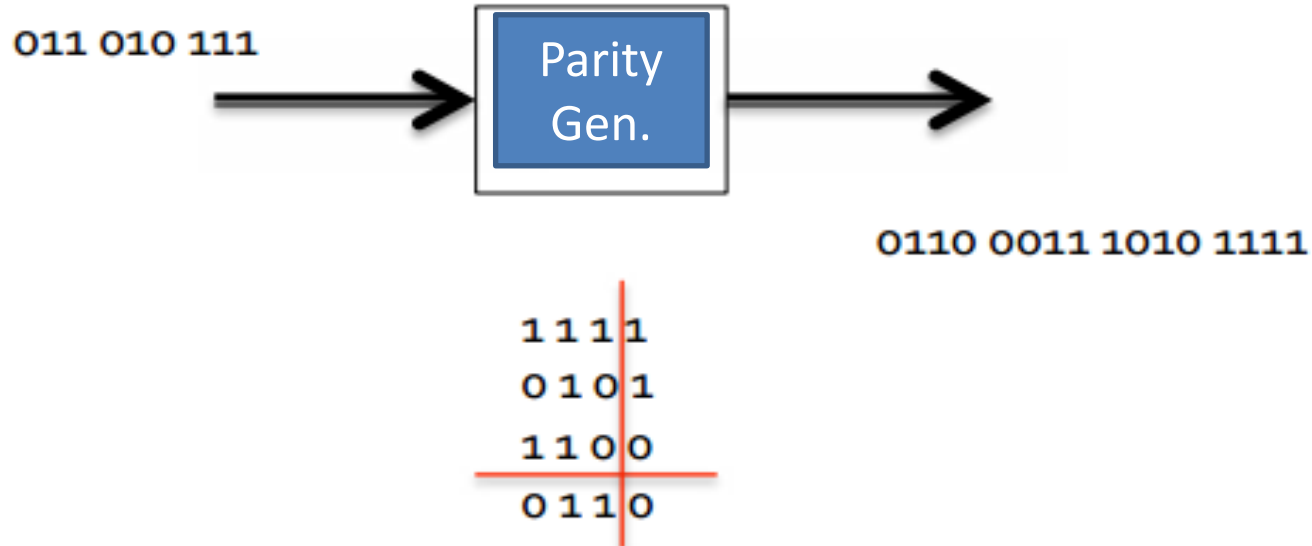
Odd → 1011011

0101110

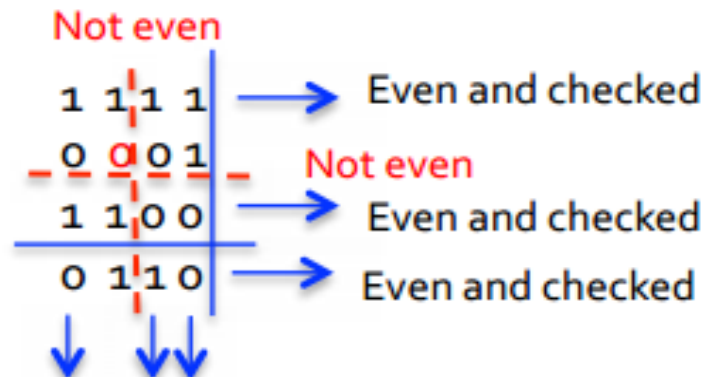
Even → 01011100

Odd → 01011101

Error correction

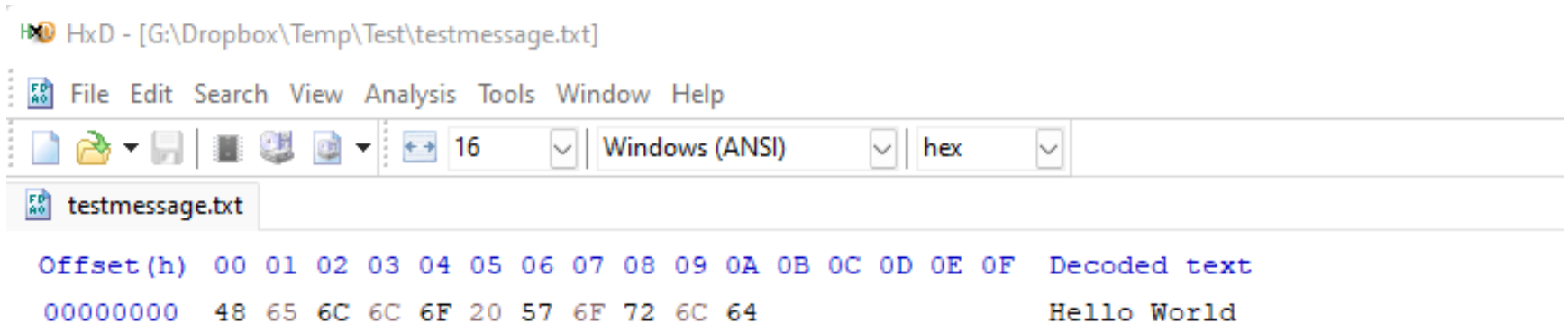


At receiver, if there is an error



TEXT DATA

Text file

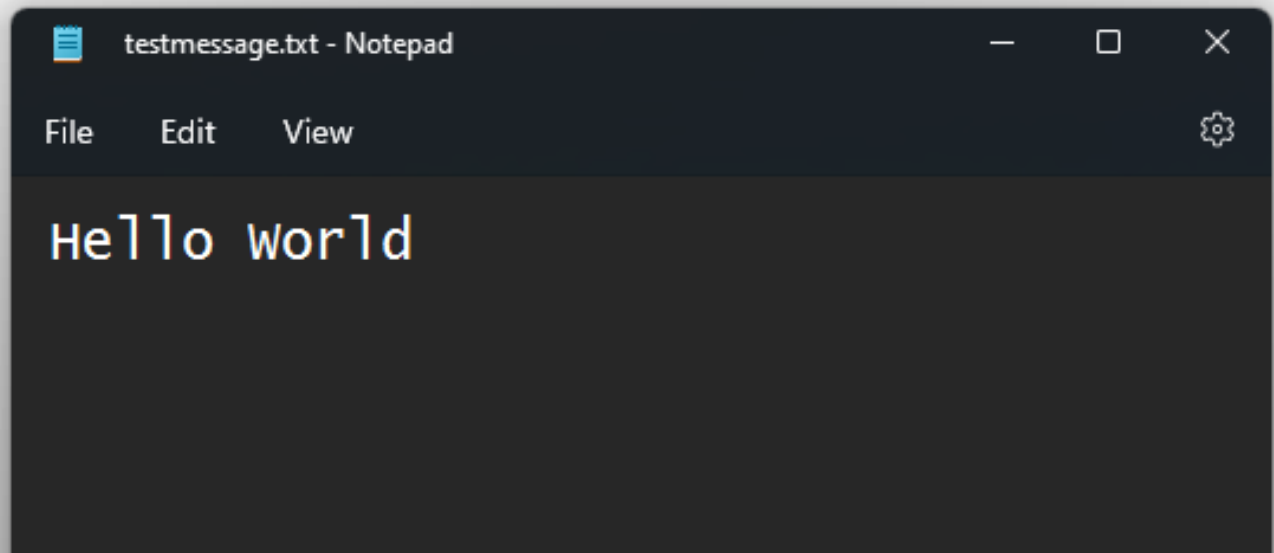


HxD - [G:\Dropbox\Temp\Test\testmessage.txt]

File Edit Search View Analysis Tools Window Help

testmessage.txt

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	48	65	6C	6C	6F	20	57	6F	72	6C	64						Hello World



testmessage.txt - Notepad

File Edit View

Hello world

American Standard Code for Information Interchange (ASCII)

[illegible]

USASCII code chart

b7b6b5b4b3b2b1Bits ColumnRow					000	001	010	011	100	101	110	111
b4↑	b3↑	b2↑	b1↑	Row↓	0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	\	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

USASCII code chart

b₇ b₆ b₅ b₄ b₃ b₂ b₁ Bits					0 0 0 0 0 1 0 1 0 0 1 1 1 0 0 1 0 1 1 1 0 1 1 1							
Column Row					0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	\	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	K	h	k
1	0	0	1	9	HT	IM	)	9	I	L	i	l
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

A = 41H = 1000001

ADDRESS BUS, DATA BUS AND CONTROL SIGNAL

ADDRESS, DATA, and CONTROL

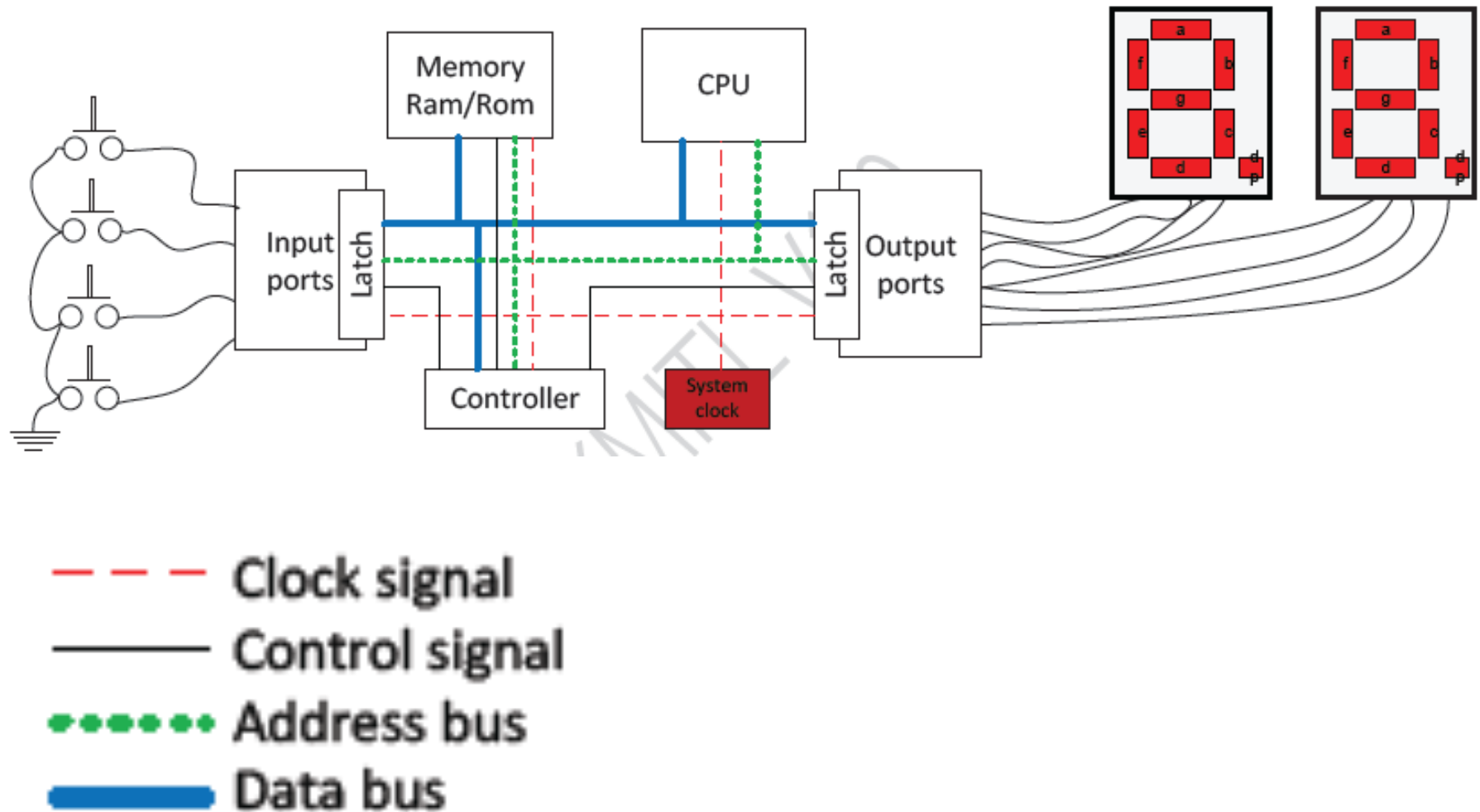
- **Address bus** is a sequence of binary data that points to the location of the data in the memory or I/O devices.
- **Control signal** is the signal sending to the devices such as the memory, the I/O to control reading, writing, enable, and disable.
- **Data bus** is the data that sending from a device to other device on the data wires. The data location relates to the address pointing the position.

Activity 2.3 Describe process of parcel express.



- What is an issue concern in the parcel post?

Data transferring in the computer system



Device Address in computer

I/O port address	Description
060H – 064H	Keyboard controller
170H – 376H	Secondary IDE hard-disk controller
1F0H – 3F6H	Primary IDE hard-disk controller
220H	Sound card
300H	Network interface controller card (LAN card)
330H	SCSI adapter
3F2H	Floppy drive controller
2E8H, 2F8H, 3E8H, 3F8H	Communication port 1-4 (COM1-4)
278H, 378H, 3BCH	Line printer terminal port 1,2 (LPT1, 2, 3)



Example transferring data in the comport with C-language

```
#include <dos.h>
#include <stdio.h>
#include <conio.h>
#define PORT1 0x3F8 /* Defines Serial Port Base Address (COM1 */
void main(void){
    unsigned char c = 0;
    unsigned char chrctr = 0;
    /*int exit = 1; */
    outportb(PORT1 + 1, 0); /* Turn off interrupts */
    /* PORT1 Communication Settings */
    outportb(PORT1 + 3, 0x80); /* Set DLAB ON */
    outportb(PORT1 + 0, 0x0C); /* Set the baud rate to 9600 */
    outportb(PORT1 + 1, 0x00); /* Set Baud - Divisor latch HIGH */
    outportb(PORT1 + 3, 0x03); /* 8 bits, no parity, 1 stop */
    outportb(PORT1 + 2, 0xC7); /* FIFO Control Register */
    outportb(PORT1 + 4, 0x0B); /* Turn on DTR, RTS, and OUT2) */
    printf("Waiting on transmission from source.\nPress ESC to quit.\n");
    while(chrctr != 27){ /* Execute the loop if ESC has been hit */
        c = inportb(PORT1 + 5);
        if (c & 0x01){
            chrctr = inportb(PORT1);
            printf("%d",chrctr);
        }
        if (kbhit()){
            chrctr = getch();
            outportb(PORT1, chrctr);
        }
    }
}
```

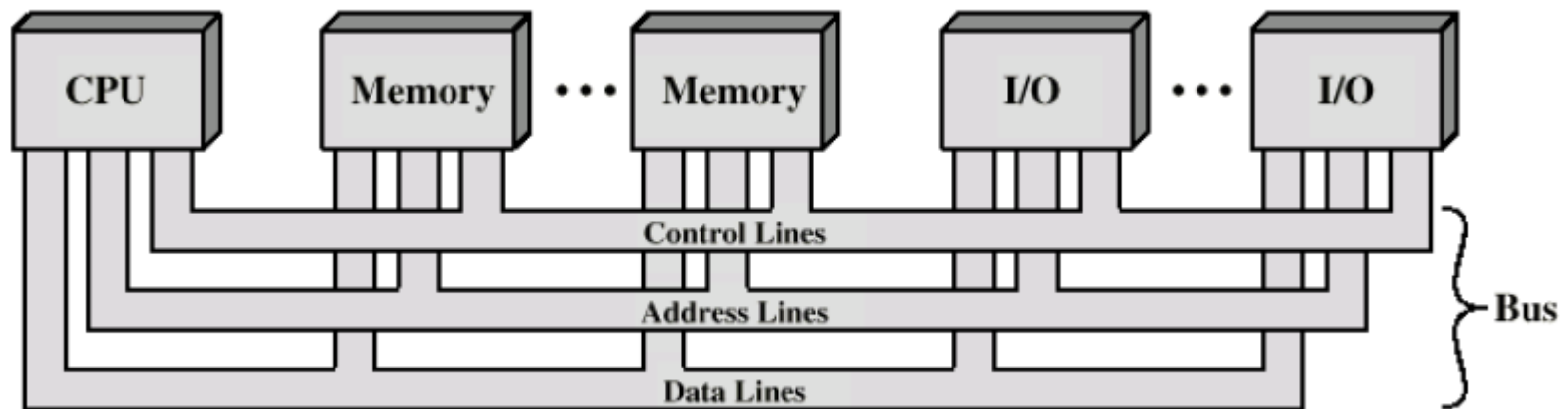
Address



Digital data in computer

A computer has three bus

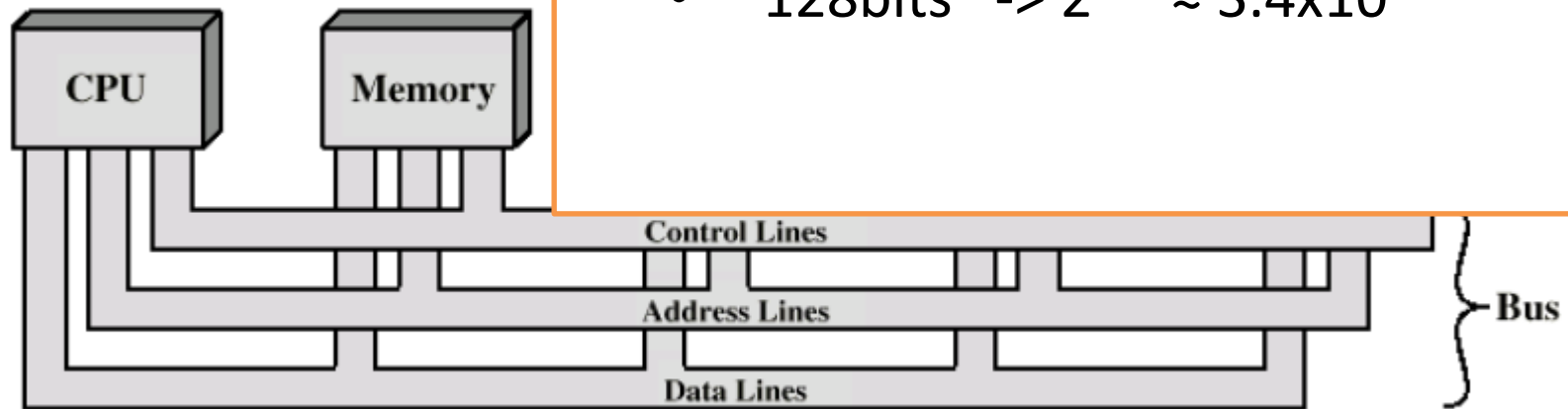
- Data bus:
- Address bus:
- Control bus:



Digital

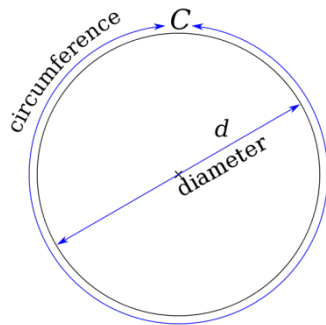
A computer has

- Data bus:
 - Address bus:
 - Control bus:
- Data bus carries data or information from CPU/a device to another device.
 - Size of data bus
 - 8bits $\rightarrow 2^8 = 256$ levels
 - 16bits $\rightarrow 2^{16} = 65,536 \approx 64K$
 - 32bits $\rightarrow 2^{32} \approx 4G$
 - 64bits $\rightarrow 2^{64} \approx 1.8 \times 10^9$
 - 128bits $\rightarrow 2^{128} \approx 3.4 \times 10^{38}$

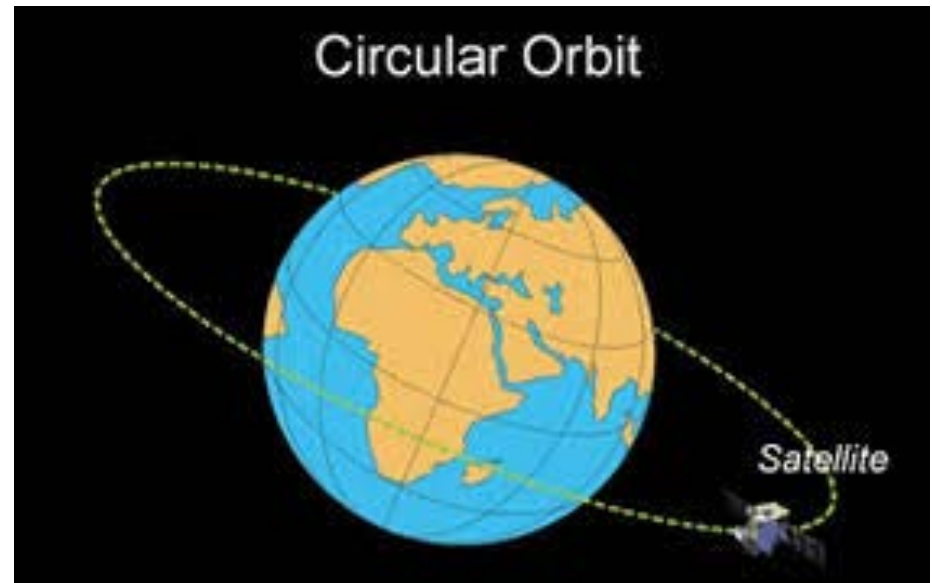


Why does the computer development from 8 bit to 128 bit?

- Calculation circumference between two objects



$$C = 2\pi r$$



Activity 2.4 Proof Float and Double keep Pi

Approximation

A quick and easy approximation for π is $22/7$

$$22/7 = 3.1428571...$$

But as you can see, $22/7$ is **not exactly right**. In fact π is not equal to the ratio of any two numbers, which makes it an irrational number.

A really good approximation, better than 1 part in 10 million, is:

$$355/113 = 3.1415929...$$

(think "113355", slash the middle "113/355", then flip "355/113")

Summary:

$$\begin{aligned} 22/7 &= 3.1428571... \\ 355/113 &= 3.1415929... \\ \pi &= 3.14159265... \end{aligned}$$

<https://www.mathsisfun.com/numbers/pi.html>

```
#include <stdio.h>

int main()
{
    float pi_f = 355.0/113.0;
    double pi_d = 355.0/113.0;
    printf("%1.50e",pi_f);
    printf("\n");
    printf("%1.50e",pi_d);
    return 0;
}
```

A computer has

- Data bus:
- Address bus:
- Control bus:

- Address bus relates to position of devices.
- For example memory 4Byte at 8bit has

Address 0

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Address 1

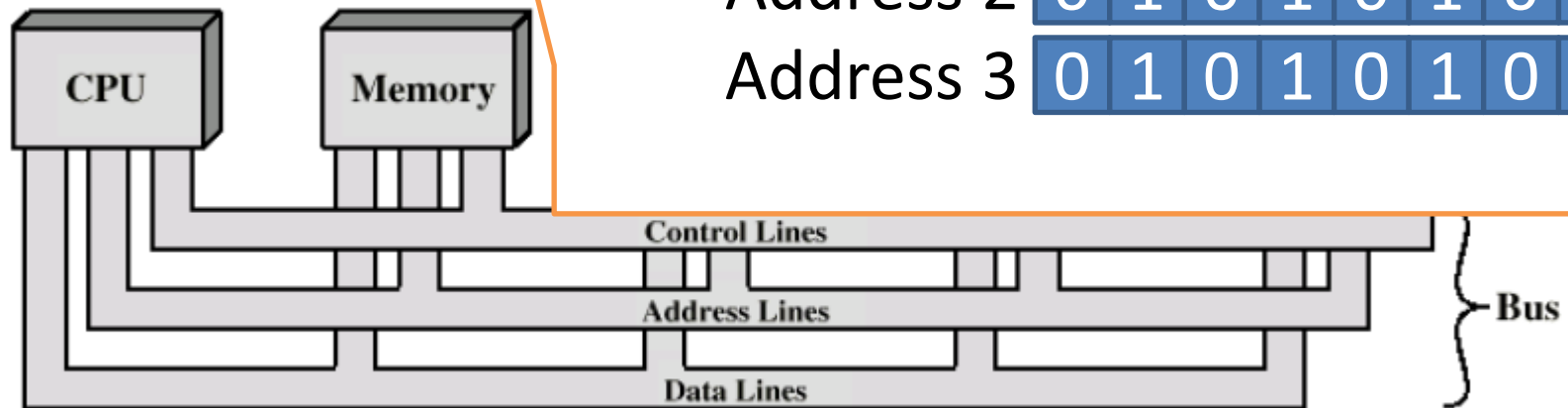
0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Address 2

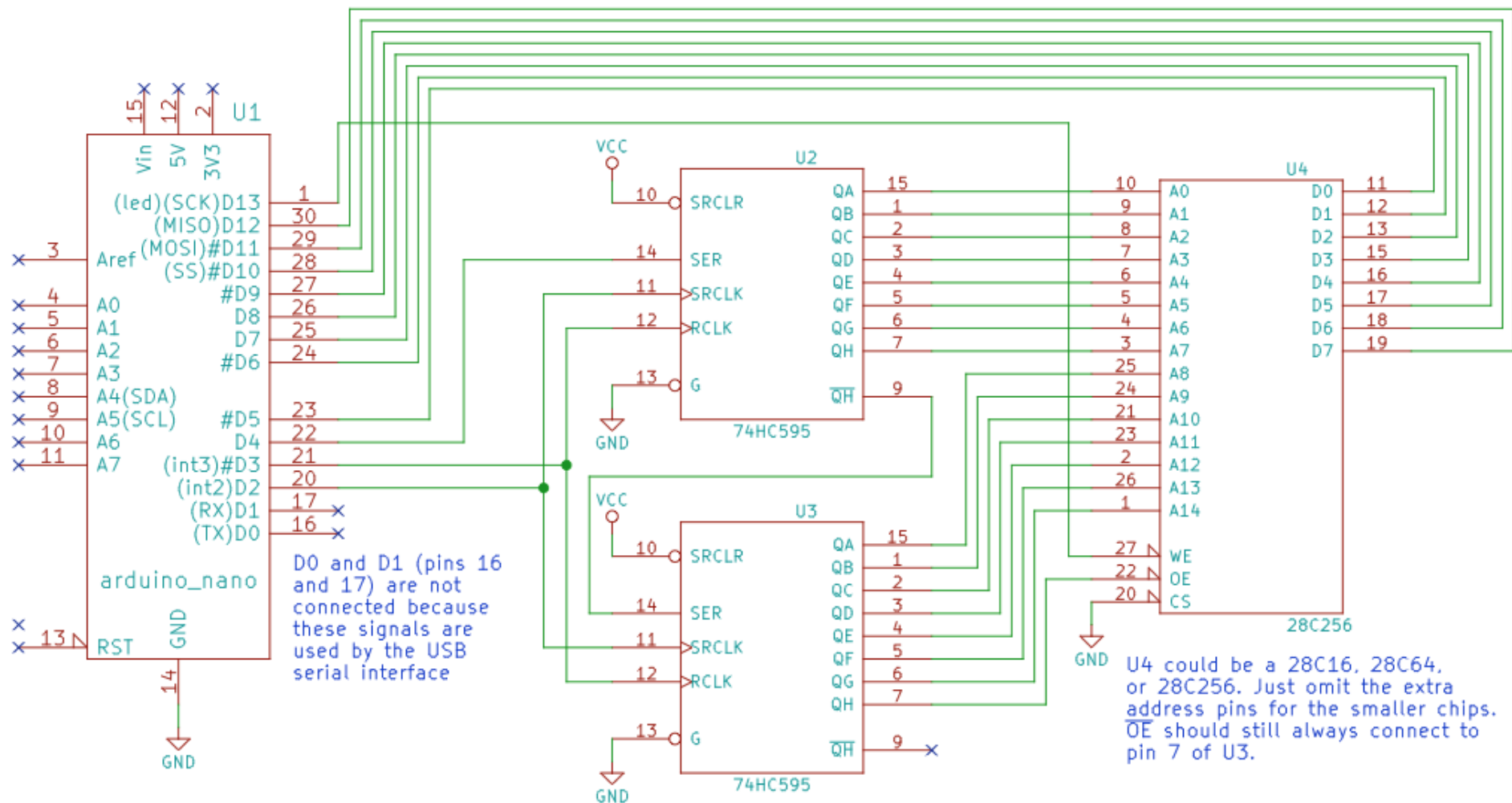
0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Address 3

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

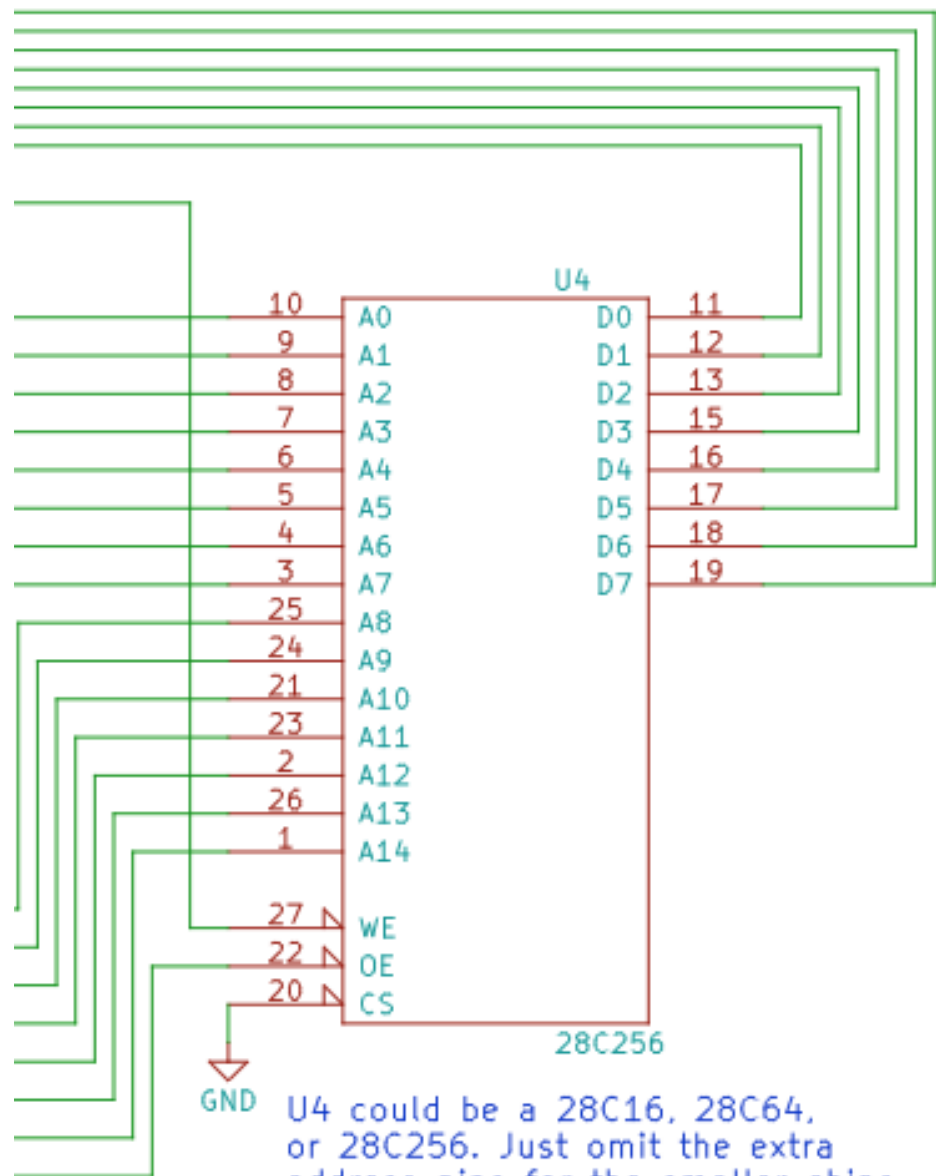


Activity 2.5, The circuit diagram shown below has 4 ICs, given you consider, which is the chip doing as the memory.

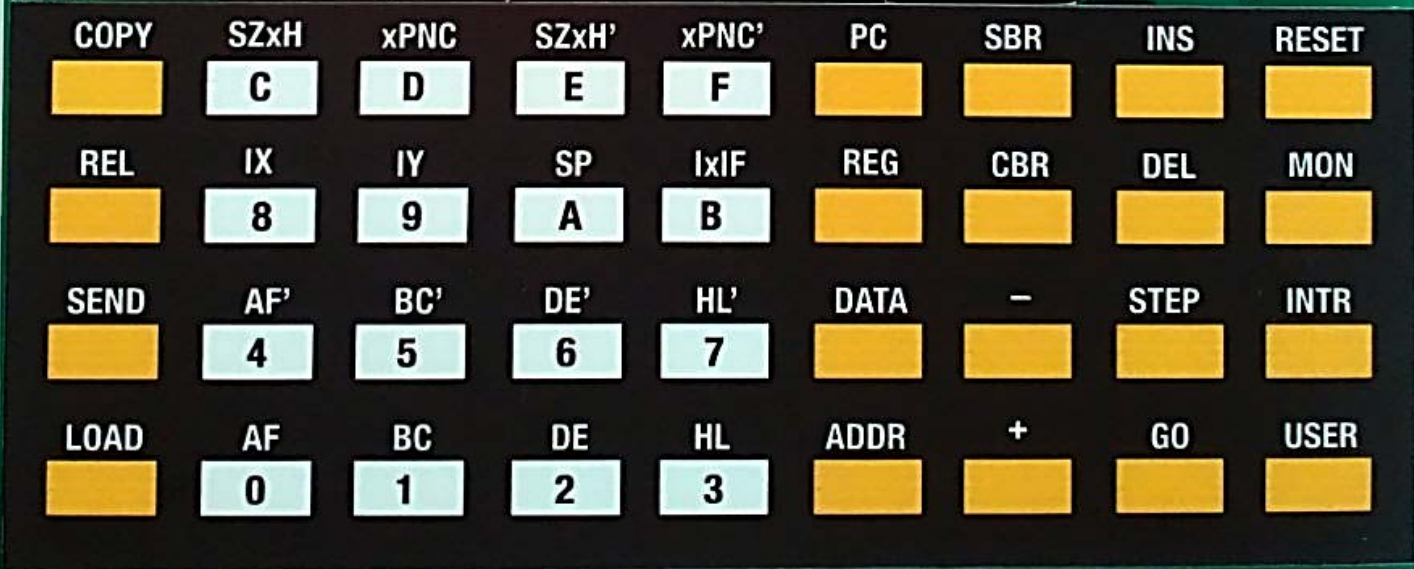


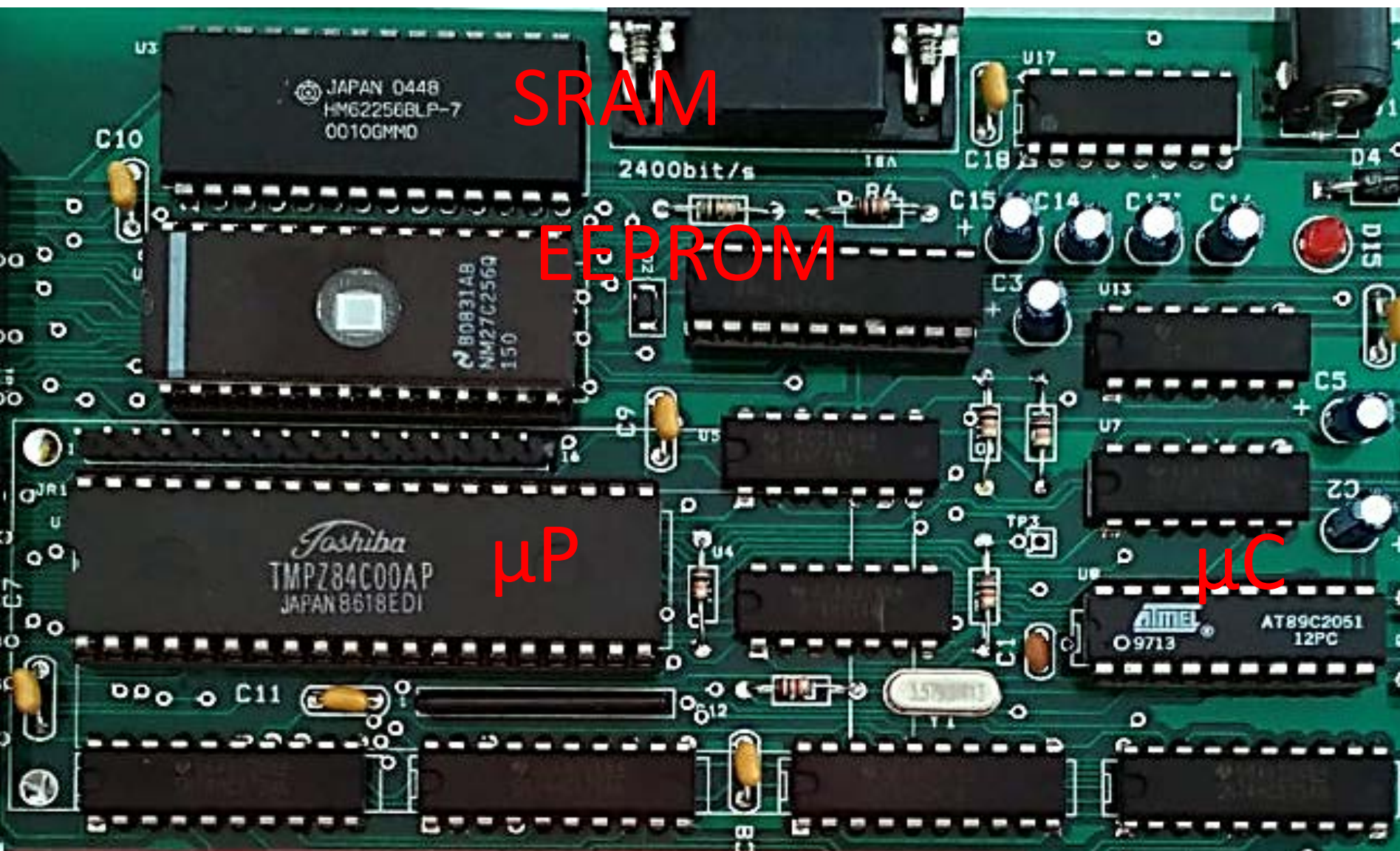
Activity 2.6, The memory IC has the pin name as the list below, given you write the full name of the pin.

- 1) An _____
- 2) Dn _____
- 3) WE _____
- 4) OE _____
- 5) CS _____

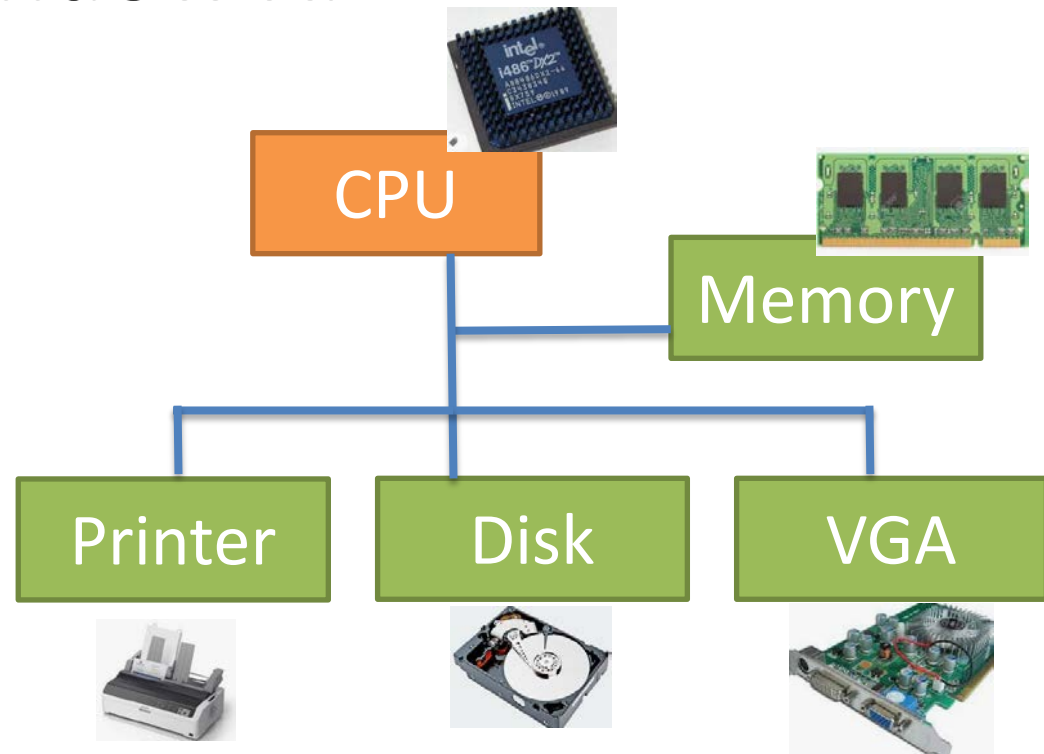
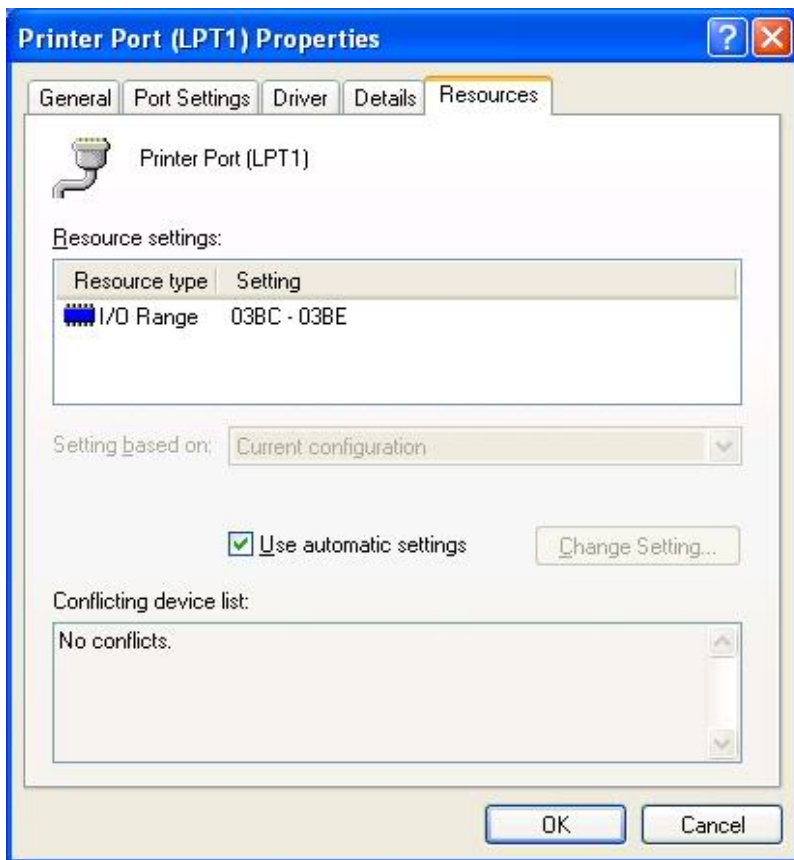


U4 could be a 28C16, 28C64, or 28C256. Just omit the extra address pins for the smaller chips. OE should still always connect to pin 7 of U3.



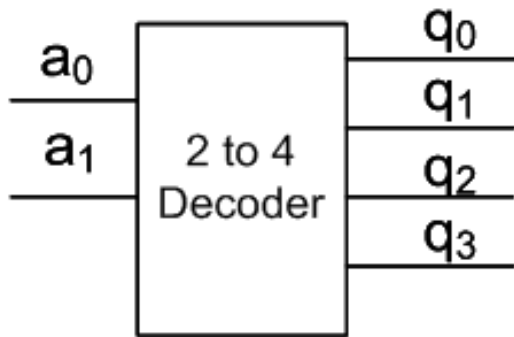


Example printer port address in Window XP

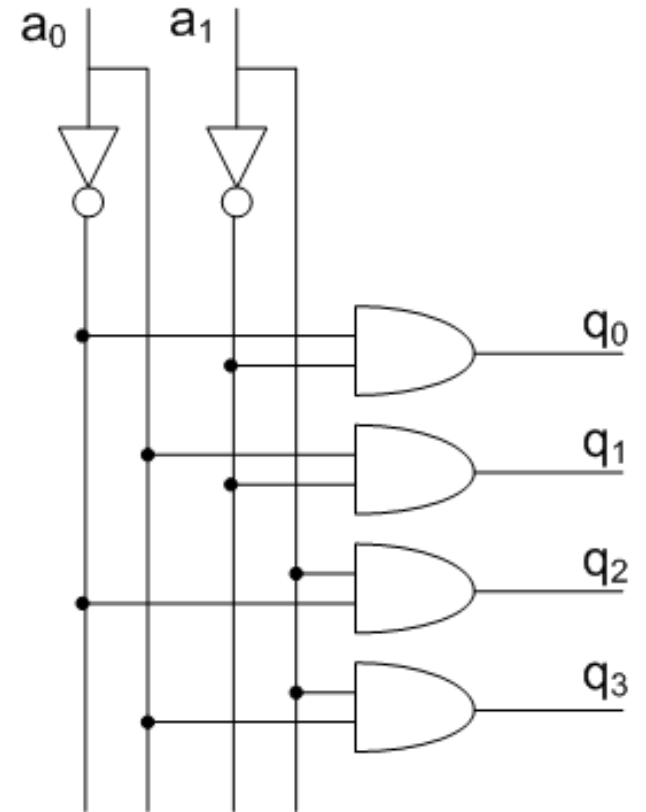


DECODER CIRCUIT

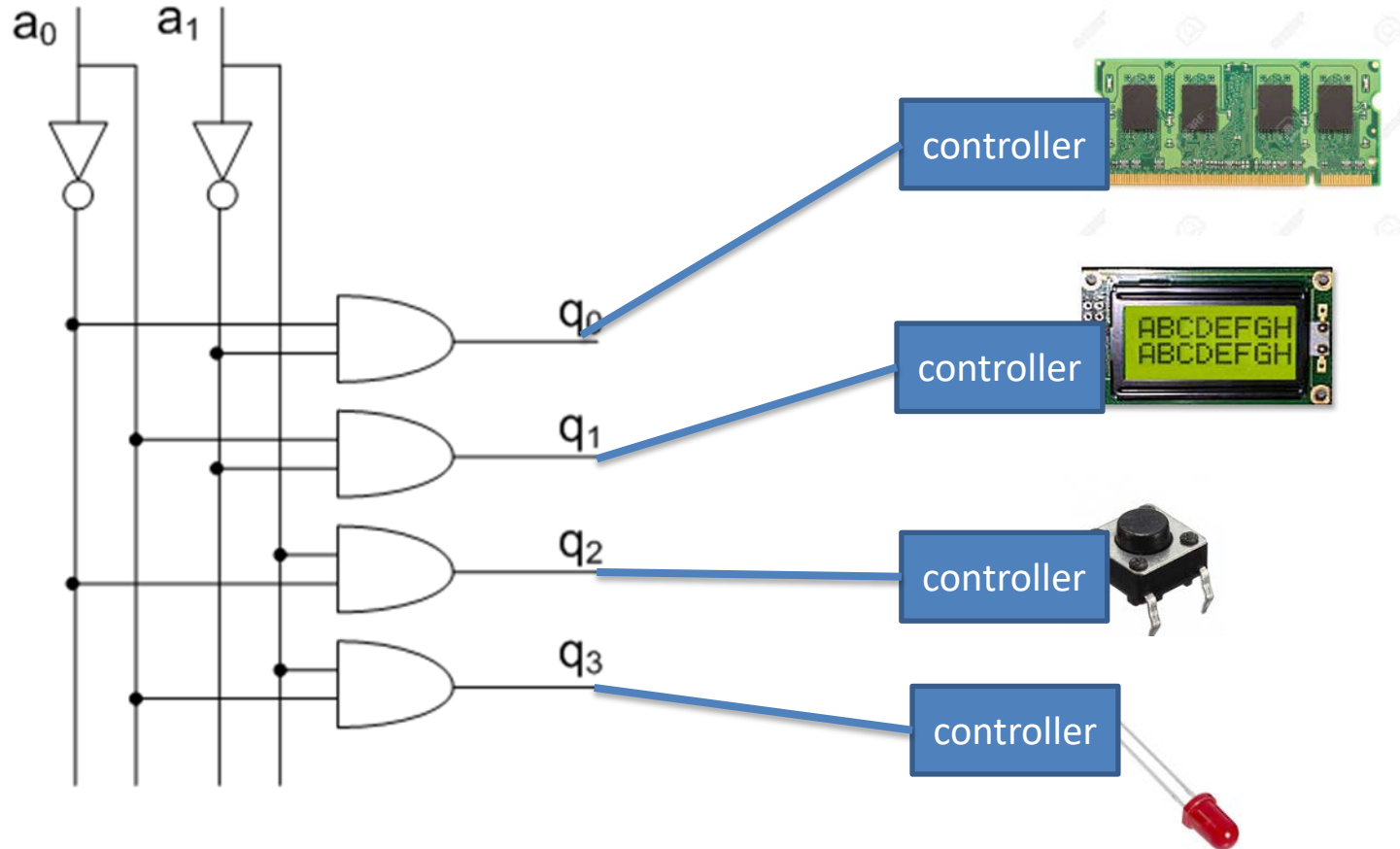
Decoder circuit



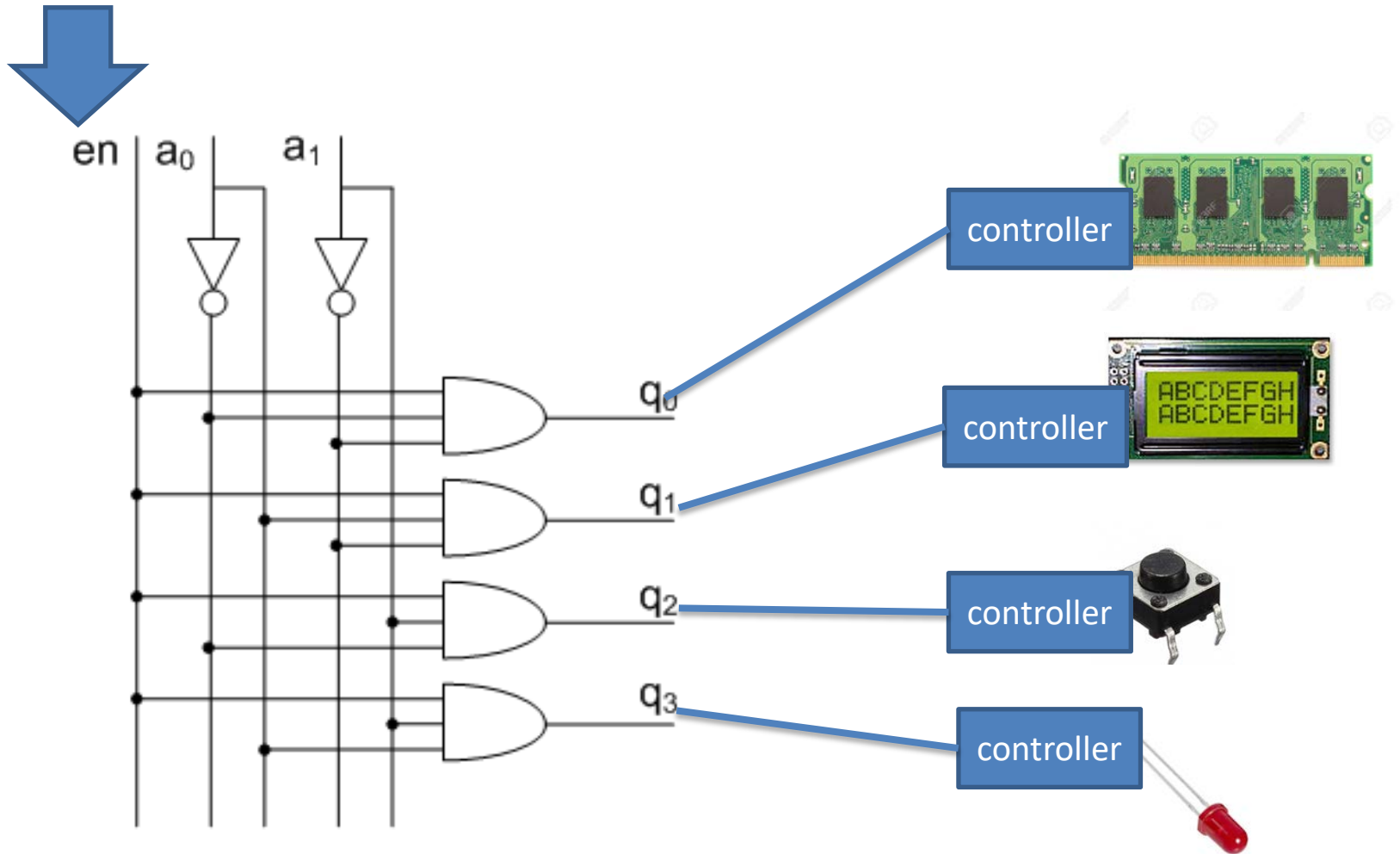
a0	a1	q0	q1	q2	q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



Decoder circuit selects devices



Decoder circuit with enable



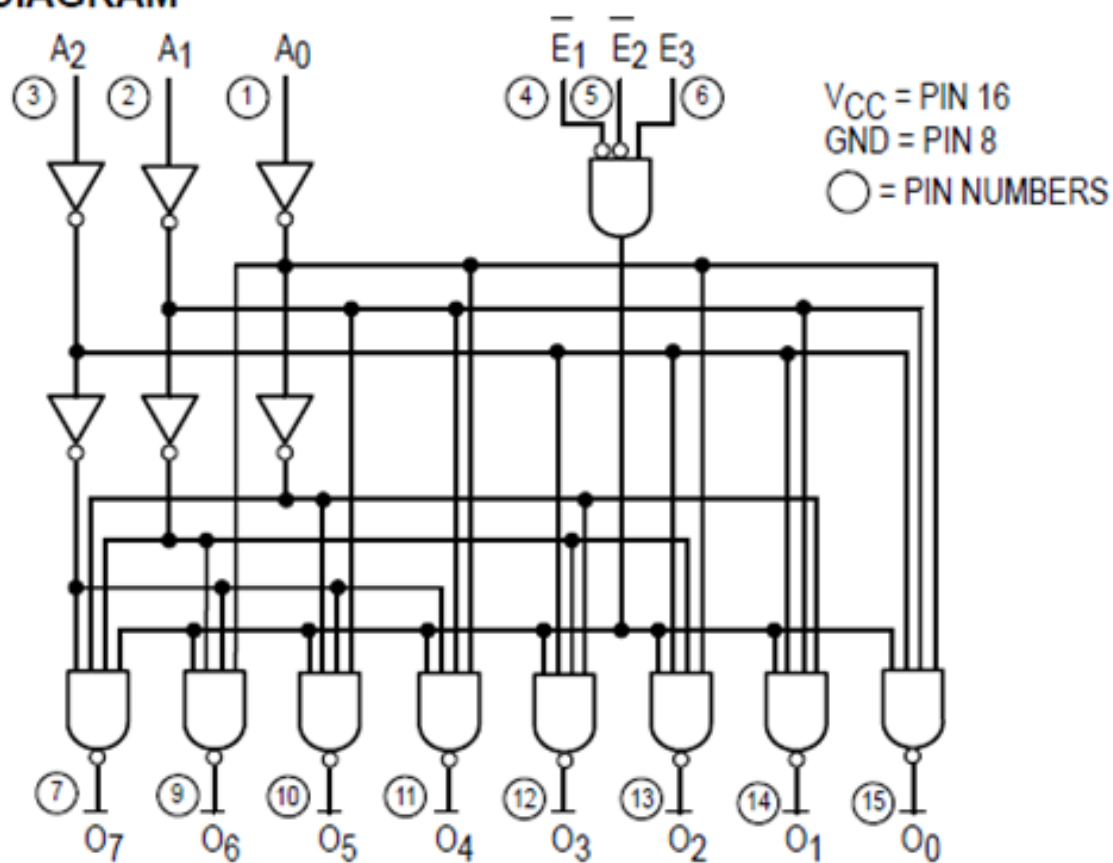


MOTOROLA

1-OF-8 DECODER/ DEMULTIPLEXER

SN54/74LS138

LOGIC DIAGRAM



TRUTH TABLE

INPUTS						OUTPUTS							
E ₁	E ₂	E ₃	A ₀	A ₁	A ₂	O ₀	O ₁	O ₂	O ₃	O ₄	O ₅	O ₆	O ₇
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

H = HIGH Voltage Level

L = LOW Voltage Level

X = Don't Care

Extended address

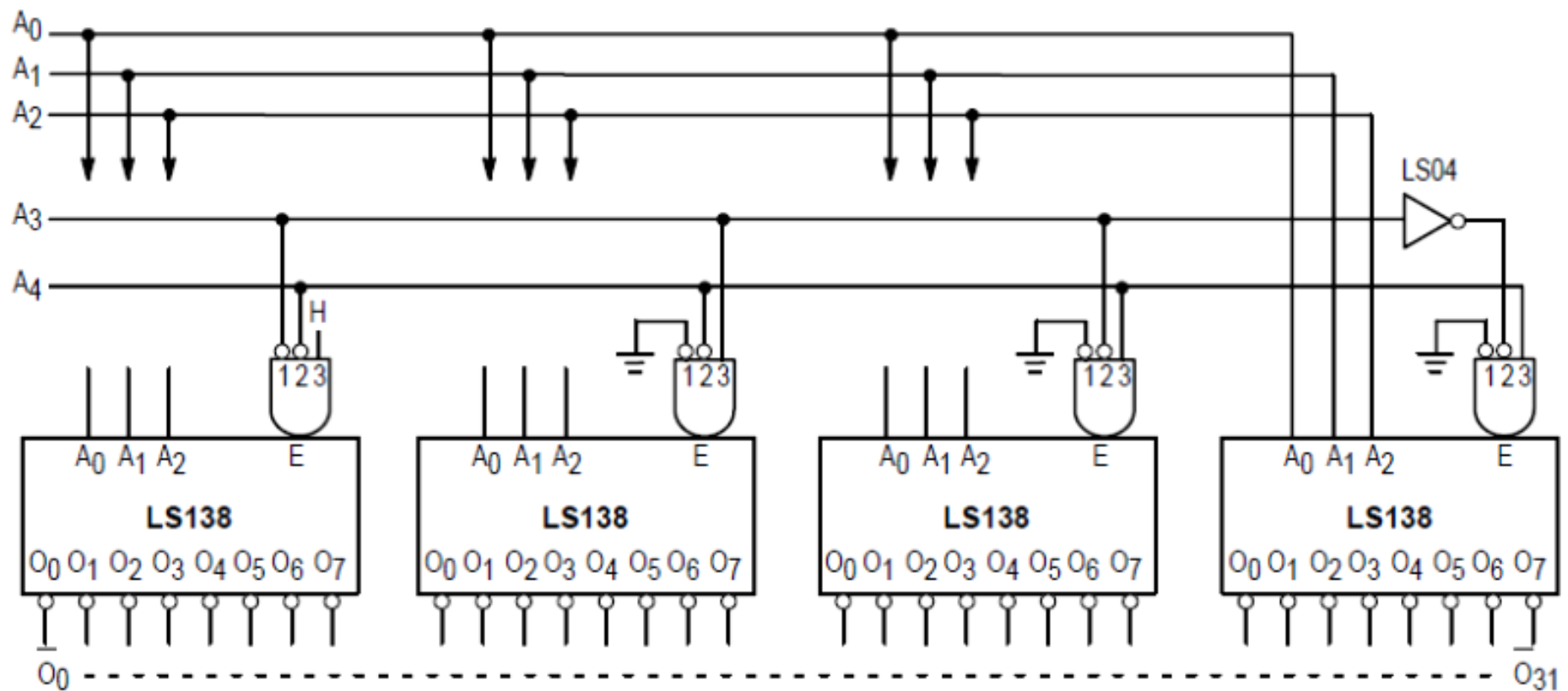


Figure a

GUARANTEED OPERATING RANGES

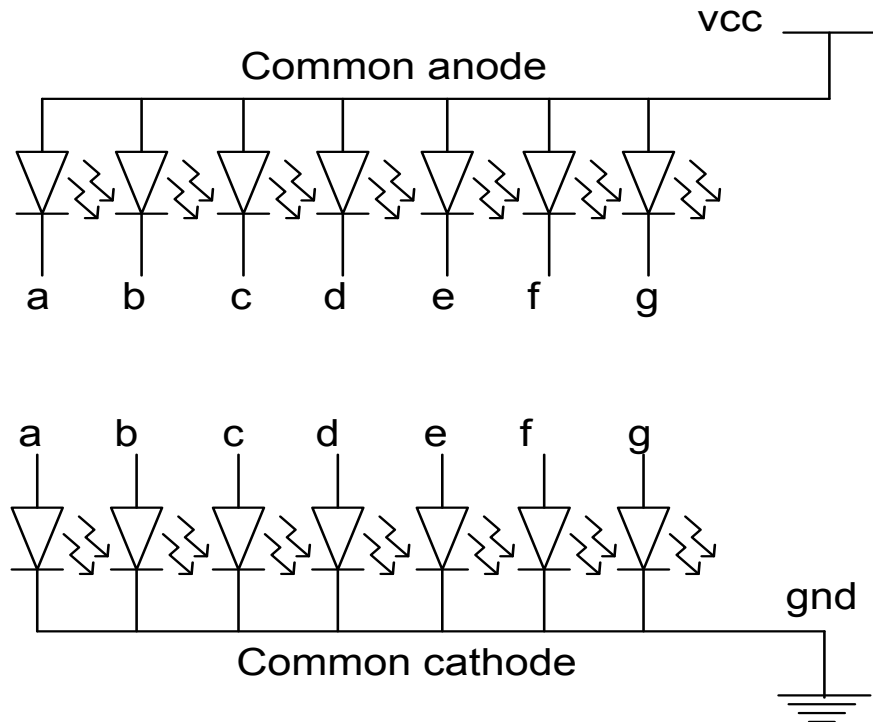
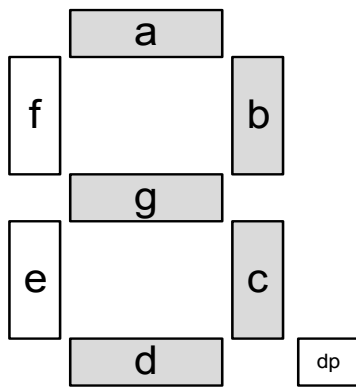
Symbol	Parameter		Min	Typ	Max	Unit
V_{CC}	Supply Voltage	54 74	4.5 4.75	5.0 5.0	5.5 5.25	V
T_A	Operating Ambient Temperature Range	54 74	-55 0	25 25	125 70	°C
I_{OH}	Output Current — High	54, 74			-0.4	mA
I_{OL}	Output Current — Low	54 74			4.0 8.0	mA

DC CHARACTERISTICS OVER OPERATING TEMPERATURE RANGE (unless otherwise specified)

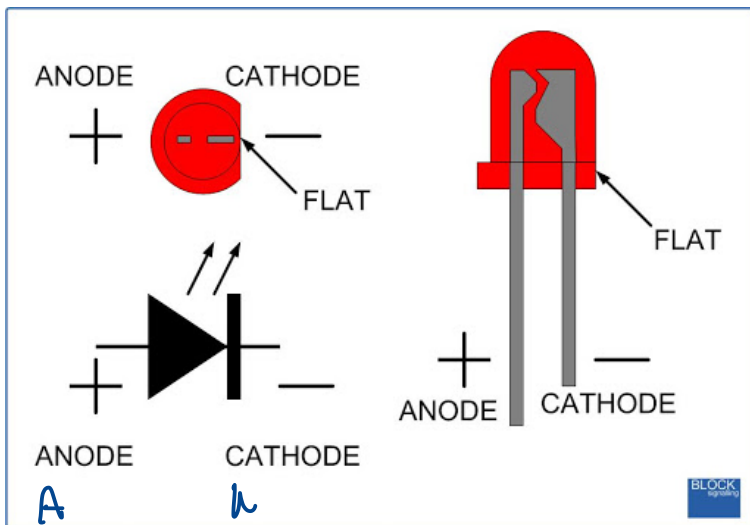
Symbol	Parameter		Limits			Unit	Test Conditions	
			Min	Typ	Max			
V _{IH}	Input HIGH Voltage		2.0			V	Guaranteed Input HIGH Voltage for All Inputs	
V _{IL}	Input LOW Voltage	54			0.7	V	Guaranteed Input LOW Voltage for All Inputs	
		74			0.8			
V _{IK}	Input Clamp Diode Voltage			−0.65	−1.5	V	V _{CC} = MIN, I _{IN} = −18 mA	
V _{OH}	Output HIGH Voltage	54	2.5	3.5		V	V _{CC} = MIN, I _{OH} = MAX, V _{IN} = V _{IH} or V _{IL} per Truth Table	
		74	2.7	3.5		V		
V _{OL}	Output LOW Voltage	54, 74		0.25	0.4	V	I _{OL} = 4.0 mA	V _{CC} = V _{CC} MIN, V _{IN} = V _{IL} or V _{IH} per Truth Table
		74		0.35	0.5	V	I _{OL} = 8.0 mA	
I _{IH}	Input HIGH Current				20	μA	V _{CC} = MAX, V _{IN} = 2.7 V	
					0.1	mA	V _{CC} = MAX, V _{IN} = 7.0 V	
I _{IL}	Input LOW Current				−0.4	mA	V _{CC} = MAX, V _{IN} = 0.4 V	
I _{OS}	Short Circuit Current (Note 1)		−20		−100	mA	V _{CC} = MAX	
I _{CC}	Power Supply Current				10	mA	V _{CC} = MAX	

LED SEVEN SEGMENT COMMON ANODE AND CATHODE

Activity: 2.7 Plot your name on 7-segment



บันทึกด้วย



Alphabet present on 7-Segment

Once we have identified the decoded letter as a 5-bit value, we would like to display it. To do so, we can use a 7-segment LED display as shown in Fig. 1. For invalid addresses, we will display a dash (i.e. only segment *G* illuminated).

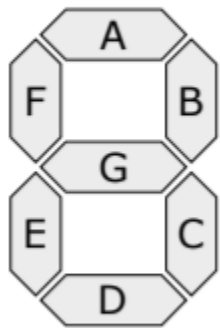


Figure 1: 7 segment display alphabet.

Summary